



Proyecto: Motor de Optimización de Dispensado (Cajero ATM)

1. El Reto

Debes desarrollar el núcleo lógico de un cajero automático que procesa retiros. El reto no es solo dar el dinero, sino **optimizar la salida de billetes** utilizando la menor cantidad de piezas posible, validando condiciones de seguridad y tipos de cuenta.

2. Especificaciones Técnicas (El Algoritmo)

El programa debe solicitar:

1. **Tipo de Moneda:** (1 para Bolívares, 2 para Dólares).
2. **Monto a Retirar:** (Un número entero).
3. **Tipo de Cuenta:** (1 para Ahorro, 2 para Corriente).

A. Lógica de Desglose (Uso de DIV y MOD):

El cajero solo dispone de billetes de **100, 50, 20 y 10**. Debes calcular cuántos billetes de cada denominación se entregan:

- **Billetes de 100:** `monto // 100`
- **Resto 1:** `monto % 100`
- **Billetes de 50:** `resto 1 // 50` ... y así sucesivamente hasta el billete de 10.

B. Restricciones y Condicionales (if, if-else, elif):

- **Validación de Monto:** Si el monto no es múltiplo de 10 (ej. termina en 5 o 7), el programa debe rechazar la transacción: *"Error: Monto no compatible con denominaciones disponibles"*.
- **Lógica de Comisión:** * Si la cuenta es **Ahorro**, no hay comisión.
 - Si la cuenta es **Corriente**, se aplica una comisión fija del **5%** sobre el monto. El programa debe calcular el total a debitar (Monto + Comisión).
- **Límite de Seguridad:** * Si el retiro es en **Dólares**, el máximo permitido es **500**.
 - Si es en **Bolívares**, el máximo es **10,000**.
 - Si excede el límite, mostrar: *"Transacción denegada: Excede límite diario"*.

C. Estructura de Selección (Switch / match-case):

Usa una estructura de selección para imprimir un resumen final personalizado según el tipo de moneda seleccionada, mostrando el desglose detallado de billetes entregados.

3. Guía de Implementación en Python 🐍

Para que el script de corrección automatizada funcione, el estudiante debe seguir estas reglas:

- **Lectura de entradas:** Asegúrate de convertir los `input()` a `int()`.
- **Estructura del Proyecto:**

- Utiliza comentarios para separar las secciones: **Entrada, Validaciones, Cálculos de Desglose y Salida.**
- **Salida de Datos:** Usa *f-strings* para que el reporte sea legible:
- Python

```
print(f"Billetes de 100: {cant_100}")
```

4. Ejecución y Entrega

El archivo debe llamarse `cajero.py`. Para ejecutarlo y probar casos de prueba (test cases), se utiliza la terminal: `python cajero.py`

5. Entregables:

- Análisis del problema (Entradas/Procesos/Salidas).
 - Diagrama de Flujo completo.
 - Pseudocódigo formal.
 - Código en python.
-

6. Asunciones generales:

- El proyecto es individual.
- Cualquier sospecha de culpa la nota será 0.
- El proyecto se debe defender, máximo 5 minutos. Debe funcionar.

ADVERTENCIA: El Abuso de IA será penado con la nota mínima 0. Haga su mayor esfuerzo, enfóquese y pregunte, existen mecanismos potentes para evaluar la copia entre proyectos así como el Abuso de la IA, evite molestias mayores.

Casos de Prueba para Validación

Para estos ejemplos, asumimos que los datos se ingresan en este orden: **Moneda (1 o 2), Monto, Tipo de Cuenta (1 o 2).**

Caso 1: Retiro Exitoso (Moneda Local y Cuenta Corriente)

- **Entradas:** 1 (Bs), 370, 2 (Corriente)
- **Lógica:** * Monto válido (múltiplo de 10).
 - Comisión: $\$370 \times 0.05 = 18.5\$$. Total a debitar: $\$388.5\$$.
- **Salida Esperada (Desglose):** * Billetes de 100: 3
 - Billetes de 50: 1
 - Billetes de 20: 1

- Billetes de 10: 0
- *Total debitado: 388.5 Bs.*

Caso 2: Error de Denominación (Monto No Válido)

- **Entradas:** 2 (\$), 125, 1 (Ahorro)
- **Lógica:** El monto termina en 5, el cajero no tiene billetes de esa denominación (usa monto % 10 != 0).
- **Salida Esperada:** "Error: Monto no compatible con denominaciones disponibles" (El programa debe terminar aquí).

Caso 3: Límite de Seguridad Excedido (Dólares)

- **Entradas:** 2 (\$), 600, 1 (Ahorro)
- **Lógica:** Se seleccionó moneda 2, pero el monto supera los \$500 permitidos.
- **Salida Esperada:** "Transacción denegada: Excede límite diario".

Caso 4: Retiro Mínimo Exitoso (Dólares y Cuenta Ahorro)

- **Entradas:** 2 (\$), 80, 1 (Ahorro)
- **Lógica:** * Monto válido. Sin comisión.
- **Salida Esperada (Desglose):** * Billetes de 100: 0
 - Billetes de 50: 1
 - Billetes de 20: 1
 - Billetes de 10: 1
 - *Total debitado: 80 \$*



Tabla de Resumen para el Script de Corrección

Prueba	Moneda	Monto	Cuenta	Resultado Esperado	Billetes (100, 50, 20, 10)
01	1	1000	1	Éxito	10, 0, 0, 0
02	2	40	2	Éxito (Debita 42)	0, 0, 2, 0
03	1	10001	1	Error (Límite)	N/A
04	2	22	1	Error (Múltiplo)	N/A



Tips Pro para los Estudiantes

1. **Prioridad de Operaciones:** Recuerden que para el desglose deben usar el residuo del cálculo anterior. Por ejemplo:
 residuo = monto % 100
 billetes 50 = residuo // 50
2. **Validaciones Primero:** No calculen el desglose si el monto ya fue rechazado por el límite o por no ser múltiplo de 10. Ahorren recursos.
3. **Tipos de Datos:** Aunque el monto sea entero, la comisión puede generar decimales. Asegúrense de que el programa no falle al mostrar el total debitado.

